6. HAFTA: Yazılım Geliştirme Yaşam Döngüsü

- ❖ Yazılım Yaşam Döngüsü Temel Adımları
- ❖ SDLC’nin avantajları ve dezavantajları





Yazılım Geliştirme Yaşam Döngüsü (SDLC) Nedir?

SDLC, bir yazılım ürününün fikir aşamasından başlayarak geliştirilmesi, piyasaya sürülmesi ve emekliye ayrılmasına kadar geçen tüm süreci kapsayan, yapılandırılmış bir metodolojidir. Tıpkı bir bina inşa ederken izlenen adımlar gibi (temel atma, kat çıkma, tesisat, boya vb.), yazılım geliştirmenin de belirli ve sıralı adımları vardır. Bu adımlar projenin kalitesini artırır, maliyetleri kontrol altında tutar ve riskleri en aza indirir.

Yazılım Yaşam Döngüsü Temel Adımları

1. Adım: Planlama (Planning)

Bu aşama, projenin "yapılabilir" olup olmadığını anlamaya odaklanan ilk ve en kritik adımdır. Fikrin masaya yatırıldığı ve projenin genel hatlarının çizildiği aşamadır.

Amacı

Projenin kapsamını, hedeflerini, kaynaklarını ve risklerini belirleyerek bir yol haritası oluşturmak.
"Bu projeyi yapmalı mıyız ve yapabilir miyiz?"
sorusuna cevap aranır.

Temel Aktiviteler

- **Fizibilite Çalışması: Projenin teknik, ekonomik ve operasyonel olarak mümkün olup olmadığının analizi.**
- **Kaynak Planlaması: Gerekli insan kaynağı (yazılımcı, analist, test uzmanı), bütçe ve donanımın belirlenmesi.**
- **Proje Takviminin Oluşturulması: İşlerin ne zaman başlayıp ne zaman biteceğine dair tahmini bir zaman çizelgesi hazırlanması.**
- **Risk Analizi: Projeyi olumsuz etkileyebilecek potansiyel sorunların (teknik, finansal, yönetimsel) öngörülmesi.**

Çıktısı: Proje Planı ve Fizibilite Raporu. Bu belgeler, projenin tüm paydaşlar için bir referans noktası olur.
Metafor: Bir ev inşa etmeye karar vermeden önce arsanın uygunluğunu kontrol etmek, bütçeyi belirlemek ve ne kadar süreceğini tahmin etmek.

2. Adım: Gereksinim Analizi (Requirement Analysis)

Planlama aşamasında "evet, bu proje yapılabilir" kararı çıktıktan sonra, "ne yapılacağı" sorusunun detaylandırıldığı aşamadır. Müşterinin ve son kullanıcıların beklentileri toplanır ve belgelenir.

Amacı

Yazılımın tam olarak ne yapması gerektiğini, hangi işlevlere sahip olacağını ve hangi kısıtlar altında çalışacağını net bir şekilde tanımlamak.

Temel Aktiviteler

- **Müşteri ve kullanıcılarla toplantılar, mülakatlar ve anketler yapmak.**
- **Mevcut sistemleri incelemek.**

Çıktısı: Yazılım Gereksinim Spesifikasyonu (Software Requirement Specification - SRS) belgesi. Bu belge, projenin anayasası gibidir ve sonraki tüm adımlar için temel oluşturur.

Metafor: Evin kaç odalı olacağı, mutfağın nerede bulunacağı, pencerelerin boyutu gibi tüm detayların mimari çizimden önce belirlenmesi.

3. Adım: Tasarım (Design)

Gereksinimler netleştikten sonra, bu gereksinimlerin "nasıl" karşılanacağının planlandığı teknik aşamadır. Yazılımın mimarisi ve iskeleti burada oluşturulur.

Yüksek Seviyeli Tasarım

Sistemin genel mimarisi, kullanılacak teknoloji yığını (programlama dili, veritabanı vb.) ve modüller arasındaki ilişkiler belirlenir.

Düşük Seviyeli Tasarım

Her bir modülün iç yapısı, algoritmalar, sınıflar ve fonksiyonlar detaylı olarak tasarlanır.

UI/UX Tasarımı

Kullanıcı Arayüzü (UI) ve Kullanıcı Deneyimi (UX) tasarımlarının yapılması.

Amacı: Yazılımın mimarisini, modüllerini, arayüzlerini, veri tabanı yapısını ve diğer teknik özelliklerini belirlemek.

- **Çıktısı: Tasarım Dokümanları, Veritabanı Şemaları, Mimari Diyagramlar ve Prototip/Wireframe'ler.**

  Metafor: Mimarın, evin tüm detaylarını içeren (elektrik tesisatı, su tesisatı, duvar kalınlıkları vb.) teknik planları ve kat planlarını çizmesi.

4. Adım: Geliştirme / Kodlama (Development / Implementation)

Tasarım aşamasının belirli bir olgunluğa ulaşmasıyla birlikte Kodlama aşaması başlar. Müşteriye teslim edilecek ürünü programlama aşamasıdır. İyi kod, okunabilirliği ve bakımı kolay olan basit koddur. KISS (Keep it simple) prensibine göre yeni mezun olmuş birisine kodunuzu verdiğinde 1–2 gün içerisinde anlayabiliyor ve değiştirebiliyorsa kodunuz iyi bir koddur.

Amacı

Tasarım dokümanlarına uygun olarak kaynak kodunu yazmak ve çalıştırılabilir bir yazılım ürünü ortaya çıkarmak.

Temel Aktiviteler

- Seçilen programlama dilinde kod yazmak.
- Veritabanını oluşturmak.
- Kodlama standartlarına ve en iyi pratiklere uymak.
- Geliştiricilerin yazdıkları küçük kod parçalarını test ettiği Birim Testleri (Unit Tests) yapmak.

Çıktısı: Kaynak Kodu ve çalıştırılabilir Yazılım Modülleri.

Metafor: İnşaat ekibinin, mimari planlara bakarak tuğlaları örmesi, duvarları dikmesi ve çatıyı inşa etmesi.

5. Adım: Test Etme / Sınama (Testing)

Yazılımın gereksinimleri karşılayıp karşılamadığını ve hatalar içerip içermediğini kontrol etmek için yapılan kritik bir aşamadır. Kalite güvence (QA) ekibi burada devreye girer.

1

Entegrasyon Testi

Farklı modüllerin bir arada uyumlu çalışıp çalışmadığını test etmek.

2

Sistem Testi

Tüm sistemin bir bütün olarak gereksinimleri karşılayıp karşılamadığını kontrol etmek.

3

Kabul Testi

Müşterinin veya son kullanıcının yazılımı test edip beklentilerini karşıladığını onaylaması.

Amacı: Yazılımdaki hataları, kusurları ve eksiklikleri tespit edip geliştirme ekibine bildirerek ürün kalitesini sağlamak.

Performans, güvenlik ve kullanılabilirlik testleri de bu aşamada yapılır.

Çıktısı: Test Raporları, Hata Listeleri ve son kullanıcıya sunulmaya hazır, doğrulanmış ve kararlı bir yazılım.
Metafor: İnşaat bittikten sonra elektrik, su tesisatının kontrol edilmesi, kapıların kilitlenip kilitlenmediğinin denenmesi ve evin genel olarak sağlamlığının test edilmesi.

6. Adım: Dağıtım / Kurulum (Deployment)

Testleri başarıyla geçen yazılımın son kullanıcılara ulaştırıldığı aşamadır. Yazılım artık "canlı" ortama taşınır.

Amacı

Yazılımı gerçek çalışma ortamına (sunuculara) kurmak ve son kullanıcıların erişimine açmak.

Temel Aktiviteler

- **Yazılımın üretim (production) sunucularına yüklenmesi.**
- **Gerekli konfigürasyonların yapılması.**
- **Varsa, eski sistemden yeni sisteme veri aktarımı (veri migrasyonu).**
- **Kullanıcıların bilgilendirilmesi ve eğitilmesi.**

Çıktısı: Çalışan ve kullanıcıların erişebildiği canlı yazılım sistemi.
Metafor: Evin anahtarlarının sahibine teslim edilmesi ve ailenin eve taşınması.



7. Adım: Bakım (Maintenance)

Yazılımın yaşam döngüsündeki en uzun ve genellikle en maliyetli aşamadır. Yazılım canlıya alındıktan sonra ortaya çıkan sorunların giderilmesi ve sistemin güncel tutulmasıdır.

Hata Düzeltme

Kullanım sırasında ortaya çıkan hataların giderilmesi.

İyileştirme

Yazılıma yeni özellikler eklemek veya mevcut özellikleri geliştirmek.

Adaptasyon

Yazılımın yeni işletim sistemleri, tarayıcılar veya donanımlarla uyumlu çalışmasını sağlamak.

Amacı: Yazılımın sorunsuz bir şekilde çalışmaya devam etmesini sağlamak, kullanıcı geri bildirimlerine göre iyileştirmeler yapmak ve değişen ihtiyaçlara uyum sağlamak.

Performans optimizasyonu ve güvenlik güncellemeleri de bu aşamada yapılır.

Çıktısı: Yazılımın yeni sürümleri (versiyonları) ve güncelleme paketleri (yamalar).

Metafor: Eve taşındıktan sonra zamanla ortaya çıkan sorunların (patlayan ampul, akan musluk vb.) tamir edilmesi veya eve yeni bir oda eklenmesi.

UYGULAMA

SDLC yaşam döngüsünü akış diyagramı ile oluşturun.



SDLC'nin Avantajları (Güçlü Yönleri)

Yazılım Geliştirme Yaşam Döngüsü (SDLC), projelere bir yapı ve disiplin getirir. Ancak her metodoloji gibi, onun da güçlü ve zayıf yönleri vardır.

1. Yüksek Kontrol ve Öngörülebilirlik

Açıklama

SDLC, projenin başından sonuna kadar izlenecek adımları net bir şekilde tanımlar. Her aşamanın belirli hedefleri ve çıktıları vardır.

Etkisi

Bu yapı, proje yöneticilerine ve ekiplere süreç üzerinde tam kontrol sağlar. Projenin hangi aşamada olduğu, ne kadar ilerlendiği ve bir sonraki adımın ne olduğu her zaman bellidir. Bu sayede belirsizlikler azalır ve proje daha öngörülebilir hale gelir.



2. Net Tanımlanmış Hedefler ve Gereksinimler

Açıklama

SDLC'nin en başında yer alan Planlama ve Analiz aşamaları, tüm paydaşları projenin hedefleri ve gereksinimleri üzerinde anlaşmaya zorlar.

Etkisi

Projenin kapsamı baştan kilitlendiği için "kapsam kayması" (scope creep) olarak bilinen, proje devam ederken sürekli yeni isteklerin eklenmesi sorunu en aza indirilir. Yanlış anlaşılmalar kodlama başlamadan önce tespit edilir ve bu da maliyeti düşürür.

3. Gelişmiş Kalite Güvencesi

01

Açıklama

Tasarım, kodlama ve test gibi aşamalar birbirinden net bir şekilde ayrılmıştır. Özellikle "Test" aşamasının başlı başına bir adım olması, kaliteye verilen önemi gösterir.

02

Etkisi

Yazılım, müşteriye ulaşmadan önce kapsamlı testlerden geçirilir. Hatalar erken aşamada yakalanır ve düzeltilir. Bu durum, son ürünün daha kararlı, güvenilir ve yüksek kalitede olmasını sağlar.

4. Kapsamlı Dokümantasyon ve Bilgi Aktarımı



Açıklama

SDLC'nin her aşamasında (Proje Planı, SRS, Tasarım Dokümanları, Test Raporları vb.) resmi belgeler üretilir.



Etkisi

Bu dokümantasyon, projenin uzun vadeli bakımı için hayati önem taşır. Ekibe yeni katılan bir geliştirici, dokümanları okuyarak sistemi hızla anlayabilir. Ekipten birileri ayrılsa bile projenin bilgisi kişilere bağımlı kalmaz, kurumsal hafızada saklanır.

5. Etkin Kaynak ve Maliyet Yönetimi

Açıklama

Planlama aşamasında projenin bütçesi, takvimi ve ihtiyaç duyulan kaynaklar (insan, donanım vb.) detaylı olarak belirlenir.

Etkisi

Bu, projenin bütçe ve zaman planına sadık kalma olasılığını artırır. Kaynakların verimli kullanılması sağlanır ve finansal sürprizlerin önüne geçilir.



SDLC'nin Dezavantajları (Zorlukları)

Bu dezavantajlar, özellikle Şelale (Waterfall) gibi geleneksel ve katı SDLC modellerinde daha belirgindir.

1. Katılık ve Değişime Uyum Zorluğu

Açıklama: Geleneksel SDLC modelleri, bir sonraki aşamaya geçmeden önce mevcut aşamanın tamamen bitirilmesini gerektirir. Bir aşama bittikten sonra geri dönüp değişiklik yapmak çok zor ve maliyetlidir.

Etkisi: Günümüzün hızla değişen pazar koşullarında bu bir sorundur. Müşterinin bir gereksinimi değiştiğinde veya yeni bir teknoloji çıktığında, bu katı yapı değişime adapte olmakta zorlanır. Proje bittiğinde, başlangıçtaki ihtiyaçlar geçerliliğini yitirmiş olabilir.

2. Uzun Geliştirme Süreleri ve Geç Geri Bildirim

Açıklama: Müşteri veya son kullanıcı, çalışan bir ürünü görmek için döngünün neredeyse sonuna kadar (Test veya Teslim aşamasına kadar) beklemek zorundadır.

Etkisi: Eğer analiz aşamasında bir şeyler yanlış anlaşıldıysa, bu durum ancak aylar sonra, her şey bittiğinde fark edilir. Bu noktada hatayı düzeltmek çok pahalıdır. Ayrıca, pazar fırsatları kaçırılabilir çünkü ürünün piyasaya çıkması uzun sürer.

3. Yüksek Başlangıç Maliyeti ve Bürokrasi

Açıklama: SDLC'nin ilk aşamaları olan planlama, analiz ve tasarım, çok fazla dokümantasyon ve toplantı gerektirir.

Etkisi: Henüz tek bir satır kod yazılmadan önce ciddi bir zaman ve kaynak harcanır. Bu durum, özellikle küçük ve orta ölçekli projeler için aşırı bürokratik ve yavaşlatıcı olabilir.

4. Gereksinimlerin Başlangıçta Mükemmel Olması Varsayımı

Açıklama: Tüm süreç, projenin en başında tüm gereksinimlerin eksiksiz ve doğru bir şekilde tanımlanabileceği varsayımına dayanır.

Etkisi: Gerçek hayatta, müşteriler genellikle ne istediklerini tam olarak bilmezler veya bir prototip görene kadar fikirleri netleşmez. Başlangıçta yapılan yanlış bir varsayım veya eksik bir gereksinim, tüm projenin yanlış temeller üzerine inşa edilmesine neden olabilir.

UYGULAMA

SDLC Kullanılmayan Bir Projede Neler Olur?

En az 5 tane problem yazın

(gerçekçi örneklerle açıklayın)



Sonuç ve Modern Yaklaşımlar

Gördüğümüz gibi, geleneksel SDLC yapıları istikrarlı, gereksinimleri net ve büyük ölçekli projeler için harika bir çerçeve sunar.

Ancak, dezavantajları (özellikle katılık ve yavaşlık), yazılım dünyasını yeni çözümler aramaya itmiştir. Haftaya işleyeceğimiz **Çevik (Agile) metodolojiler** (Scrum, Kanban vb.), bu sorunları aşmak için doğmuştur. Çevik yaklaşımlar, SDLC'nin temel adımlarını daha küçük, tekrarlayan döngülerle (sprint'ler) ele alarak esneklik, hızlı teslimat ve sürekli geri bildirim sağlamayı hedefler.